

Learning Outcomes: In this lab, you will:

- Use the pandas library to read and analyze CSV data.
 - Use the matplotlib library to produce meaningful visualizations.
 - Continue using the prettytable library to pretty-print tabular data to the screen.
 - Reinforce concepts in writing flexible Python functions.
 - Continue to use git and GitHub.
-

Getting Started:

1. Create a GitHub repo named similar to dcs211_f2025_lab4_student1_student2, and add me as a collaborator.
2. If you don't already have the prettytable, pandas, and matplotlib modules installed, use pip to do so:

```
python3 -m pip install prettytable
python3 -m pip install pandas
python3 -m pip install matplotlib
```

3. From Lyceum, download the county_economic_status_2024.csv file and save into your working directory for this lab. This CSV file was obtained by directly saving the “CountyEconomicStatusandDistress” tab from the Excel file containing Fiscal Year (FY) 2024 data on county economic status, available from the Appalachian Regional Commission (ARC):

<https://www.arc.gov/map/county-economic-status-in-appalachia-fy-2024>

Motivation/Context: This lab will be about learning to use pandas for data analysis, but you will also be doing some reflection on the data you are analyzing. Create a new Google Doc into which you will add answers and reflections to various prompts throughout this lab. You will need to submit a PDF of this reflection document with your lab submission.

The data for this lab is associated with the Appalachian Region, an economically challenged region of the US. To start, read more about the Appalachian Region (where I am originally from):

<https://www.arc.gov/about-the-appalachian-region/>

and then respond in your Google Doc to the following prompts.

1. How is the Appalachian Region defined? How many US States are included?
 2. Provide some context on how rural Appalachia compares to the rest of rural America, specifically, in:
 - population and demographics
 - education
 - employment
 - income and poverty
 - broadband access
 3. Briefly describe each of the five listed goals in the ARC's Strategic Plan for investing in the Appalachian Region (<https://www.arc.gov/strategicplan/>).
-

Inspecting the Data: Open the provided CSV in Excel or Numbers and look at the data you will be working with. Also notice the format of the table. Based on your pandas tutorial work, you should note a few things:

- There are several descriptive rows at the top of the CSV that you will want to ignore when you read the data in. (NOTE: For this assignment, you are not permitted to modify the CSV file directly — you will have to account for those rows in your Python implementation.) Make note of the row on which the data (specifically, column headers) begins.
- There are also a few rows at the bottom of the CSV that you will want to ignore when reading the data in.
- The column names provided by the ARC, while descriptive, will be cumbersome to work with in Python — you will eventually want to rename those columns.

There will be three pieces of data that you will focus on in this lab: (three-year) average unemployment rate, per capita market income, and poverty rate. In your Google Doc, provide summary definitions for each of those three pieces of data. (If you download the original Excel file from the ARC — use the first link provided in this document — you can find their definitions under the “Source & Methodology” tab. Make sure to summarize in your own words — don’t just copy-and-paste.)

Also provide an answer to how the “poverty threshold” is defined, and cite your source.

Coding/Analysis Work: As you complete the work of implementing a Python script, I encourage you to work alongside in the interpreter, testing each element along the way so you can see how particular items are working (or not working!).

1. All your code should go inside one file named `dcs211_lab4.py`.
2. Remember to import the libraries you will need for this work: `pandas`, `matplotlib.pyplot`, and `PrettyTable`.
3. Use `pandas` to read the provided CSV file. There are arguments to `read_csv` that will be useful to you: `skiprows`, `skipfooter`, `engine`, and `thousands`. See the API for details. Again, I encourage you to work in the interpreter until you’re sure your data frame looks to be in a reasonable form.
4. Rename the columns using more succinct labels than those provided by the ARC. To do so, you can use code similar to:

```
arc_data.columns = ['fips', 'state', 'county', 'arc_county', ...
```

essentially replacing the `columns` field with your own list of shorter (but still meaningful) labels.

5. Look at your data frame in the interpreter. The first row of data is empty, no? Remove that row (just use slicing, with `iloc` since slices work with numbers), overwriting the data frame.
6. Include print statements (with meaningful text) to print the mean, standard deviation, minimum, and maximum poverty rate over all data in the CSV. Remember to use `pandas` methods to easily compute those stats.
7. In your Google doc, provide meaningful definitions (in your own words) of what the mean and standard deviation represent.
8. Recall that you can grab the states only using `arc_data['state']` (presuming you renamed the associated column with `'state'` above. Use the Python `type` function in the interpreter to determine the type of the result from `arc_data['state']`.
9. Use the `pandas` method named `value_counts` (which works on a `Pandas Series` object) to determine/print the number of counties per state. (Hint: Texas has 254, while Delaware has 3.)
10. Use the `PrettyTable` library to print a table of the top-ten states (using your approach from the previous item) in terms of number of counties. Include the state name, number of counties, mean per-capita income, median per-capita income, and poverty rate. Your table should look something like the following:

State	# counties	PCI (mean)	PCI (median)	Poverty Rate
Texas	254	40438.00	36698.00	15.04
Georgia	159	30384.98	27570.00	18.33
...				

Hint: To display numeric data in your table to two decimal places, use f-string formatting. For example, an f-string to format the contents of a variable number to two decimal places would look like `f"{number:.2f}"`.

- Go back in your Python program and remove the `thousands` option when reading the CSV. What is the result of running your program, and why is that option needed? Include the answer in your Google Doc.
- Same as the previous table, but include the bottom-ten states in terms of number of counties. Exclude the District of Columbia.
- Now include code to print a `PrettyTable` displaying information of the top ten counties (regardless of state) by decreasing poverty rate. Include the state, county, per capita income, poverty rate, and average unemployment. Your table should look like:

State	County	PCI	Poverty Rate	Avg Unemployment
South Dakota	Todd	20603.00	59.00	5.50
South Dakota	Oglala Lakota	18455.00	50.50	8.80
...				

- It would be very helpful to display tables of, say, the top ten or bottom ten, and based on poverty rate, per capita income, or average unemployment. Rather than copy-and-paste code, you will write a function to streamline this approach.

Write a function named `printTableBy` having the following parameters:

- the overall pandas data frame
- a field (column name, a string) to select by (e.g., `'poverty_rate'`)
- how many to include (an integer), e.g., 10 for top-ten
- a title (string) to display just before the `PrettyTable`

Your function should sort data based on the given field, and then in a `PrettyTable`:

- print the top how-many based on that field, sorted by decreasing order (e.g., decreasing poverty rate)
- print a separator (see the `divider` argument when adding a row in `PrettyTable`)
- print the bottom how-many based on that field, sorted by increasing order (e.g., increasing poverty rate).

Include the same table columns as in the previous step.

So, for example, were I to call your function as follows:

```
printTableBy(arc_data, 'poverty_rate', how_many = 3, title = "COUNTIES BY POVERTY RATE")
```

then the resulting table should look like:

COUNTIES BY POVERTY RATE				
State	County	PCI	Poverty Rate	Avg Unemployment
South Dakota	Todd	20603.00	59.00	5.50
South Dakota	Oglala Lakota	18455.00	50.50	8.80
South Dakota	Mellette	29340.00	50.00	5.10

Texas	Loving	100614.00	1.20	1.10	
Texas	Kenedy	36326.00	1.80	4.90	
Utah	Morgan	55412.00	1.80	2.60	

Notice that the top three are sorted by decreasing poverty rate, while the bottom three are sorted by increasing poverty rate. Hint: I have used f-string width formatting for the state and county (left-justified, each width 20) so that all tables will have the same widths for comparison.

Use your function to print tables for, say, the top/bottom-ten (or twenty) based on each of poverty rate, average unemployment, and PCI.

15. In your Google Doc, reflect on what you observe in your data tables. What do you notice about top vs. bottom poverty rates? Average unemployment? PCI? Are there counties that seem to be in conflict even within the same state?

Do some investigation (e.g., Wikipedia or other online resources) about these same-state counties that seem in conflict, and provide some information about why this might be the case. Cite your source(s).

16. In this last piece of the lab, you need to write a function to create a per-state bar plot using `matplotlib.pyplot`. Specifically, write a function named `createByStateBarPlot` having five parameters:

- the ARC data as a pandas data frame
- a field (column name, a string) to select by (e.g., `'poverty_rate'`)
- a filename (string) into which your figure will be saved
- a title (string) to appear at the top of your figure
- a y-label (string) to appear on the left of your figure, which will characterize the data labels appearing on your left axis.

Your bar plot should group by state (refer back to the class notes for help on using `groupby`), compute the mean of the given field, sort in increasing order, and then produce a bar plot with appropriate labels.

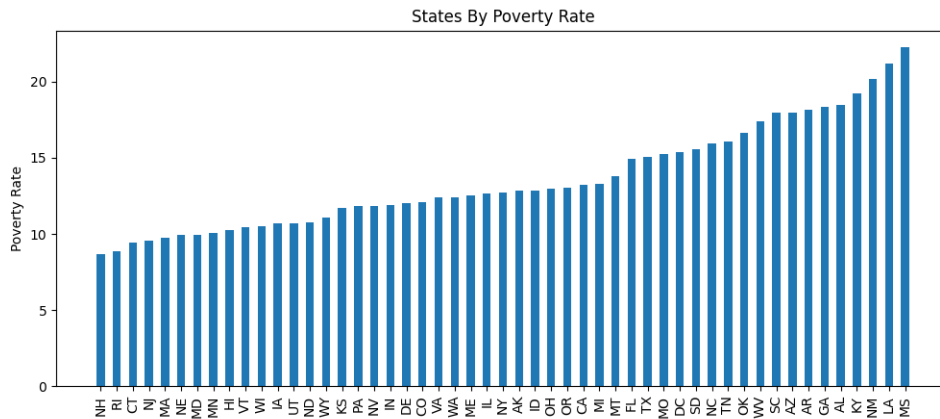
This work will be similar to the `matplotlib` work from your tutorial, but with some additional investigation required on your part. From the `matplotlib.pyplot` library, the following functions will be of use to you — see the API and other online resources:

- `plt.bar`
- `plt.figure` (for setting figure width)
- `plt.title`
- `plt.ylabel`
- `plt.xticks` (also note the rotation argument)
- `plt.savefig` (for saving to a file)

If I were to call your function as follows:

```
createByStateBarPlot(arc_data, 'poverty_rate', "pov_rate.png", "States By Poverty Rate", "Poverty Rate")
```

then the resulting bar plot saved to `pov_rate.png` should look like the following:



Notice that the x-axis on the figure includes state abbreviations. If you look at your groupby result, it will instead have state names — however, state names won't be able to fit, so you will need to replace those names with abbreviations instead.

Fortunately, someone has already done the work for creating an appropriate dictionary for us: <https://gist.github.com/rogerallen/1583593>

You can copy-and-paste that dictionary into your code, or you could be a little fancier and use `requests` to grab the text of the dictionary (use the URL resulting from “raw” button on that page instead) and then use Python's `exec` function on the response text to automatically assign the dictionary variable for you. (We will have talked about this at the start of the lab.)

After you have done your groupby, computed the mean, and sorted, just overwrite that structure's `.index` with a matching list of state abbreviations (similar to what you did for `.columns` on the original CSV). **It is up to you to make sure that order of your list is correct, and that along the x-axis, the state abbreviations match what would have been the state name.**

Make sure that your function produces a correct bar plot into a PNG file whether you select by poverty rate or by average unemployment or by per capita income. (I will be calling your function under all three of these scenarios, and will expect to see three different, correct bar plots.)

Submitting: Make sure that your completed `dcs211_lab4.py` file resides on the shared GitHub repo, and also include there the PDF of your reflection/responses. Make sure that your two functions are well-documented, using appropriate docstrings and type hints.